RHK-CADcam — Project Review

10 July 2026 · Full codebase: 200 source files, ~59,000 lines (app/ + src/) · Five parallel subsystem readers (canvas editor, resolver/core libs, optimiser/G-code, library screens, cross-cutting) + manual verification

Overall verdict

> The architecture is fundamentally sound and better than most projects that grow this fast (136 commits in ~10 weeks). The layering is genuinely good: the resolver and optimiser are pure, framework-free libraries with one-directional data flow (DB → CabinetInput → ResolvedCabinet → parts → nest → G-code), UI never leaks into them, and Next 16 conventions are followed correctly.
>
> The two real problems: **(1) nothing protects the correctness of machine output** — no tests, no CI, and several silent-failure paths in exactly the code that decides where holes get drilled; **(2) load-bearing geometry math is copy-pasted in up to five places** and has already started to drift.

What's solid (no action needed)

- Clean resolver/optimiser layering; pure compute cores with no React/UI leakage
- Central auth via `proxy.ts` (Next 16 middleware) + `allowed_emails` allowlist — **verified working** (one subagent reported "no auth guard"; false alarm — it looked for the old `middleware.ts` name)
- RLS-only Supabase access — no service key anywhere, no secrets committed
- Strict TypeScript with rare `any` (~33 hits in 59k lines)
- `src/lib/partOps/enums.ts` as a single vocabulary source
- The config-driven CRUD pattern in `MaterialsClient` (worth spreading — see Priority 3)

PRIORITY 1 Protect the machine output

Failures here become miscut sheets or missing joint holes with no warning.

1.1 Wipe-then-reinsert syncs have no transaction

`seamDrillSync.ts:189`, `masterSlaveSync.ts:56`, `persistResolved.ts:31` all delete generated rows, then insert. A failure between the two steps permanently loses that cabinet's drilling until the next successful sync — and a G-code run in that window silently omits the holes.

> **Fix:** move each delete+insert into a single Postgres function called via `supabase.rpc()`. Same for the Stage 6 save loop (`Stage6Gcode.tsx:181` — sequential inserts can leave an orphaned `nest_job` with partial sheets).

1.2 Master/slave coincidence tolerance is zero

`fireMasters.ts:26` sets `COINCIDENCE_EPS = 0`, but stored geometry is rounded to 0.01mm. Sub-millimetre float drift makes a physically-required joint silently not fire (only a `noTouch` counter increments).

> **Fix:** set the epsilon to match store rounding (~0.01mm), with a test proving a 0.01mm-offset master still fires.

1.3 Sync failures are swallowed

Per-cabinet errors in both sync loops (`seamDrillSync.ts:221`, `masterSlaveSync.ts:84`) only `console.error` and continue — Stage 6 then generates G-code from stale rows with no operator warning.

> **Fix:** collect failures into the returned `warnings` so `drillNotes` in Stage 6 shows "cabinet X drills failed to regenerate".

1.4 No validation at the DB boundary

`loadCabinetInput.ts` coerces rows with bare `Number(...)` — a null column becomes a `NaN` part dimension that flows straight to CNC tables.

> **Fix:** a `finiteNum()` guard in the row mappers, plus a cheap pre-persist assertion that every resolved part has finite, positive DX/DY/DZ.

PRIORITY 2 Build the safety net

2.1 Add Vitest + CI

There is no test runner, no `test` / `typecheck` script, and no `.github/` — nothing stops a type error landing. Three orphaned test files already exist but can't run: `src/lib/optimiser/fireMasters.test.ts`, `src/lib/wallPolyline.test.ts`, `src/lib/resolver/test.ts` (a manual console script).

> **Fix (~one session):** install Vitest, convert those three, add `"test"` and `"typecheck": "tsc --noEmit"` scripts, add a GitHub Actions workflow running lint + typecheck + test + build on every push.

2.2 Golden-file G-code tests

`gcode.ts` (826 lines — datum transforms, modal compaction, arc lead-in/out, onion-skin, gang drilling bitmasks) has zero tests. This is the literal machine output.

> **Fix:** snapshot-test `generateSheetGcode` for a few known sheets/machine profiles (different datum corners, gang vs single drilling, onion-skin on/off). Also unit-test the pure coordinate transforms in isolation: the rotated-drill `toSheet` (`drills.ts:65`), `projectToFrame` / `toCabinet` round-trips, and `computeGangs` spindle bitmask math (`gangDrill.ts:124`).

PRIORITY 3 Pay down the duplication debt

3.1 Elevation projection math exists in 5 files

`caseElevRect` / `tkElevRect` / `zoneElevRect` / `shelfElevRect` / `drawerBoxPartElevRect` / `slideElevRect` / `wallCabTopFor` / `cabBottomZ` are duplicated across `ElevationSVG.tsx:27-110`, `ElevationPrintSVG.tsx:30-83`, `SectionSVG.tsx:66-75`, `ElevationReportSVG.tsx:9-21`, with the canonical copies in `cabinetEditSvgHelpers.tsx`. **They already disagree:** the shared `shelfElevRect` (`cabinetEditSvgHelpers.tsx:155`) lacks the inner-drawer/pull-out/divider handling the live canvas copy has (`ElevationSVG.tsx:45-67`) — so screen and printed drawings can differ today.

> **Fix:** consolidate into one module (keep the switch-based version); import everywhere.

3.2 The projection/inversion core is triplicated

`toCabinet` is copy-pasted in `partOpProject.ts:106` and `fireMasters.ts:146`; the per-normal thickness/frame derivation exists three times (`partOpProject.ts:71-101`, `fireMasters.ts:123-141`, `partFootprint.ts`). The forward path applies mirroring the inverse deliberately omits — a latent sign-error source held together by comments.

> **Fix:** extract one shared `toCabinet` + frame builder; make the mirror convention a single documented, tested function.

3.3 Other duplications (same recipe: extract once, import)

- **3D box helpers** — `Room3DScene.tsx:64-97` carries "MUST stay identical to Cabinet3DView" as a comment contract with `Cabinet3DView.tsx:43-72`. Move next to the already-shared `caseBox`.
- **Shelf geometry twice in the resolver** — `resolveInternal.ts:135-177` vs `fittings.ts:95-168` (also the two tree walkers at `resolveInternal.ts:207-283` vs `:311-339`).
- **Four copies of a `new Function()` expression evaluator** — `jointDrilling.ts:40`, `slideDrilling.ts:64`, `doorProfile.ts:31`, `partFormula.ts`. Executes DB-stored strings as live JS; replace with one safe parser extending the existing `calc.ts` approach.
- **~500 lines of CRUD scaffolding** across 14 library/settings clients — extract `<LibraryHeader>`, `useTableRows`, `confirmDelete`, a shared save-on-blur `patchField`; spread `MaterialsClient`'s config-driven `FieldConfig` pattern.
- **Duplicate construction defaults** — `defaults/constructionMethod.ts:49` parallels `resolver/types.ts` `DEFAULT_RULES`; unify.

PRIORITY 4 Structure, performance, robustness

4.1 CanvasClient re-render storm

`CanvasClient.tsx` (~2,100 lines, ~65 state hooks) re-renders the entire editor on every mouse move because `setMouseWorld` fires per pointer-move (:801) for a footer readout (:1880).

> **Quick fix:** move `mouseWorld` to a ref + tiny subscriber component; wrap `CanvasSVG` in `React.memo`.
> **Bigger split (own session):** group interaction ephemerals into a reducer + context to kill the 50-prop signatures on `CanvasSVG` / `ElevationSVG` / `Room3DScene`. Also: `visibleResolvedParts` (:180-184) rebuilds the whole Map on any single-cabinet change — memoize per cabinet.

4.2 Placement logic in 5 places

The flip/occupied/free-slot/anchor sequence is copy-pasted at `CanvasClient.tsx:597, :859, :874, :1060, :1928` (paste already omits `fitFreeSlot` where placement uses it).

> **Fix:** one `computePlacement()` helper returning `{ wall, pos_x, pos_y, flip, fitDx }`.

4.3 Failed saves look like successes

~169 `console.error`-only sites app-wide; optimistic UI updates never roll back on DB failure; ~38 queries discard the error return entirely (including the server loaders `app/page.tsx:48,55`, `app/canvas/[roomId]/page.tsx:17,26`). `canvasDB.ts` (74 `.from()` calls) mixes both styles.

> **Fix pattern-wise, not site-by-site:** promote the `run()` mutation wrapper already written in `CabinetsLibraryClient.tsx:69-73` into a shared util with a toast surface; have `canvasDB` writes return `{ ok, error }` and roll back optimistic state on failure.

4.4 Undo does a full-table diff

`useCanvasHistory.ts:38-85` JSON-stringify-compares every wall/cabinet/benchtop and issues per-row writes on each undo/redo. Order-sensitive equality over-writes unchanged rows. Worth revisiting when touching undo.

Quick wins (small, do anytime)

Item	Where
Throw on missing Supabase env instead of placeholder-URL fallback	`src/lib/supabase.ts:6-9`
Add `.env.example` documenting required vars	repo root
Unguarded child-row deletes (no confirm)	`JointsClient.tsx:343`, `SchedulesClient.tsx:458,464`, `DoorSystemClient.tsx:403,514`
Delete/dev-guard the prototype route + duplicated untyped `ShapeEditor.jsx`	`app/shape-editor-test/`, `z_CAD_tool/`
Move `Box` type out of `PartViewer` (lib→components layering inversion)	`jointDrilling.ts:9`
Replace boilerplate README (still says port 3000; app runs 3001)	`README.md`
Move the 22 `z_*` scratch/spec folders under `docs/`	repo root
Delete dead imports	`CanvasClient.tsx:26-29`, `canvasTypes.ts:8-9`
Add `loading.tsx` for library/settings routes	`app/library/*`, `app/settings`
Skip embedded refetch when Settings already has data (double-fetch)	`SettingsClient.tsx` + each `*Client` `reload()`

Suggested order of attack

1. **Vitest + CI + the three orphaned tests** (~one session; unlocks everything else)
2. **The sync-transaction / epsilon / warning fixes** (P1.1–1.3 — small diffs, highest physical risk)
3. **Golden-file G-code tests** (P2.2)
4. **Elevation-math and projection-core consolidations** (P3.1–3.2)
5. **The error-feedback wrapper** (P4.3)
6. Structural work (CanvasClient split, CRUD extraction) opportunistically as those areas are touched

Appendix — per-subsystem notes

A. Canvas editor (`app/canvas/[roomId]/`, ~30k lines)

- Server/client split is correct: `page.tsx` (RSC, parallel loads) → `CanvasClient.tsx` god-orchestrator.
- Cabinet *inputs* (`walls` / `cabinets` / `benchtops` arrays) and cabinet *resolved parts* (`Map` in `useMaterialColours.ts:42`) are two parallel stores synced by hand in every mutation handler.
- `CabinetEditModal` hosts ~12 tabbed sub-views, each independently fetching from Supabase (27 of ~30 component files import `supabase` directly despite `canvasDB.ts` existing as the service layer). Consolidate the 5 `dbLoad*` effects (`CabinetEditModal.tsx:233-239`) into one loader; pass data down.
- Two overlapping window-keydown effects (`CanvasClient.tsx:292-341` and `:346-365`) with different dep sets — stale-closure hazard.
- `confirmDeleteWall` (`CanvasClient.tsx:374-385`) issues two awaited deletes with no error check — a failed cabinet delete still proceeds to the wall delete.

B. Resolver & core libs (`src/lib/`)

- `loadCabinetInput.ts` is an 862-line god function (~550-line main body) — split into per-domain loaders alongside the already-extracted `loadHingeInputs` / `loadKickRunInput`.
- DX/DY/DZ semantics deliberately flip per module (case: DX=depth; face zone: DX=height; drawer box: DX=depth). Documented but fragile; `persistResolved.ts:101-104` hand-maps `edge_band_front = edge_band.right` — silent, untested convention mapping.
- `resolveHinges.ts:121` trusts DB JSON blindly (`manual as number[]`); no check that hinge positions fall within the door span.
- `resolveCabinetFromDB.ts:150-152` runs persistence detached (fire-and-forget); the in-memory `persistLocks` mutex can't prevent cross-tab races.
- Magic numbers: `cab.DX > 1200` warning (`resolver.ts:78`), hinge keep-outs (`hingeClash.ts:18-20`).
- benchtop-resolver is clean and self-contained.

C. Optimiser / G-code (`src/lib/optimiser/`)

- Pipeline: normalize snapshot → select/qty → nest (skyline + annealing) → project drills live → G-code. Parts are a read-only snapshot; only Stage 6 writes (`nest_jobs` / `nest_sheets` / `nest_placements`).
- `normalize.ts:81,96` — the synthetic `source_part_key` must exactly match `part_operations.source_part_key`; drift silently drops hand-ops (comments acknowledge this).
- `types.ts:120` — material group key uses unrounded float thickness; `18` vs `18.0001` split into different sheet groups. Round it.
- `seamDrillSync.ts:87` — `as never` cast on key lookup; a mismatch silently drops holes into an aggregate `skipped` counter.
- `gcode.ts:18` documents that helical/ramp/scrap entry moves are **not** collision-checked against neighbouring parts — real gouge risk on tight nests. Either widen per-material margins to cover `ramp_in_distance` / `tool_entry_offset` or validate approach moves.
- Gang drilling and routing use two independent datum-transform code paths from one machine profile (`gcode.ts:480` vs `:249-254`) — no cross-check that they agree.

D. Library / settings screens (`app/library/*`, `app/settings`)

- Consistent two-file pattern (server page → `'use client'` client with save-on-blur per-field updates). ~9,100 lines across 14 clients; ~450-550 lines extractable.
- Silent-failure hotspots: `MaterialsClient.tsx:360-386` (generic save, console-only), `DrawerBoxesClient.tsx:154,184` (create/duplicate), `BenchtopMethodsClient.tsx:66` / `ConstructionMethodsClient.tsx:188` (fire-and-forget deletes).
- Numeric validation is silent coercion to `null`; only `name.trim()` is ever validated — negative depths/diameters accepted.
- `app/settings/page.tsx:21` — `.single()` on `shop_settings` errors if the row is missing/duplicated; handled as silent `null`.

- `layout.tsx:15-18` metadata still says "Create Next App".

E. Cross-cutting

- Next 16.2.6 / React 19.2.4 / Tailwind 4 / three r184; conventions correctly followed (`proxy.ts`, awaited `params` / `cookies()`).
- 106 files are `'use client'`; essentially one server action (`app/login/actions.ts`). Moving writes behind server actions would centralize validation and shrink the bundle — long-term direction, not urgent.
- `next.config.ts` is empty — no security headers.
- `TESTING.md` is a manual-QA ledger, mostly "To test" — the de-facto test plan is fully manual today.
- `.env.local` correctly gitignored; `ANTHROPIC_API_KEY` stays server-side.

Generated by Claude Code (Fable 5) — five parallel Explore subagents + manual synthesis and verification. Findings are evidence-based (file:line) but not exhaustively adversarially verified; treat line numbers as approximate anchors — they will drift as files change.